Fundamentos de Testing de Software

Curso avanzado para estudiantes de Ingeniería Informática - 7° semestre

Made with GAMMA

Sesión 1: Introducción al Testing de Software

Objetivos de Aprendizaje

- Distinguir entre testing manual y automatizado
- Comprender la importancia del testing en el ciclo de desarrollo
- Identificar cuándo aplicar cada tipo de testing
- Reconocer los beneficios y limitaciones de cada enfoque

Contenidos Programáticos

Este curso aborda los fundamentos del testing de software desde perspectivas teóricas y prácticas, incluyendo tipos de pruebas, metodologías, métricas y técnicas avanzadas de programación para testing.

Testing Manual

Ejecución de pruebas por humanos sin herramientas automatizadas. Ideal para casos exploratorios y validación de interfaz.

- Flexibilidad en ejecución
- Detecta errores visuales
- Mayor costo temporal

Testing Automatizado

Pruebas ejecutadas mediante scripts y herramientas especializadas. Perfecto para regresión y pruebas repetitivas.

- Ejecución rápida
- Consistencia garantizada
- Mantiene validez con cambios

Caso Práctico: Testing en Industria

En **Spotify**, el equipo de QA combina ambos enfoques: testing manual para nuevas características de interfaz (UX/UI) y automatización para validación de funcionalidad core. Realizan ~5000 pruebas automatizadas diariamente, pero mantienen testers manuales para validar experiencias de usuario antes de cada release.

Preguntas de Reflexión

1. ¿Por qué crees que Spotify mantiene testers manuales si tiene 5000 pruebas automatizadas?
2. ¿Qué tipo de errores podría detectar un tester manual que un script automatizado no?
3. ¿Cuándo recomendarías inversión inicial en automatización versus testing manual?

Conclusiones

El testing manual y automatizado son complementarios, no excluyentes. La elección depende del contexto, recursos disponibles y prioridades del proyecto.

Made with GAMMA

Sesión 2: Unit Tests - Pruebas Unitarias

Objetivos de Aprendizaje

- Definir y aplicar pruebas unitarias efectivas
- Comprender el concepto de cobertura de código
- Utilizar frameworks como JUnit, pytest o NUnit
- Identificar unidades testables en código

Desarrollo de Conceptos

Las **pruebas unitarias** verifican la funcionalidad de unidades individuales de código (métodos, funciones, clases). Son el primer nivel de testing y deben ejecutarse rápidamente, idealmente en milisegundos.

01

Aislar la unidad

Prueba un solo componente sin dependencias externas

02

Definir casos de prueba

Entradas, salidas esperadas y condiciones de borde

03

Ejecutar y validar

Verifica que el resultado coincida con lo esperado

04

Medir cobertura

Evalúa porcentaje de código cubierto por pruebas

Caso Práctico: Sistema de Pagos

En un sistema de pagos de **PayPal**, la función `calcularComision(monto)` tiene pruebas unitarias para: monto = 0 (comisión 0), monto pequeño (comisión mínima), monto grande (comisión máxima), y casos extremos. La cobertura de esta función es del 100%, con 15 casos de prueba diferentes ejecutándose en 200ms.

Preguntas de Reflexión

1. ¿Qué ventajas ofrece la cobertura del 100%? ¿Es siempre necesaria?
2. ¿Cómo manejarías el testing de funciones con dependencias externas (APIs, bases de datos)?
3. ¿Por qué es importante que las pruebas unitarias sean rápidas?

Conclusiones

Las pruebas unitarias son la base del testing efectivo, garantizando que cada componente funcione correctamente de manera aislada antes de integrarlos.

Made with GAMMA

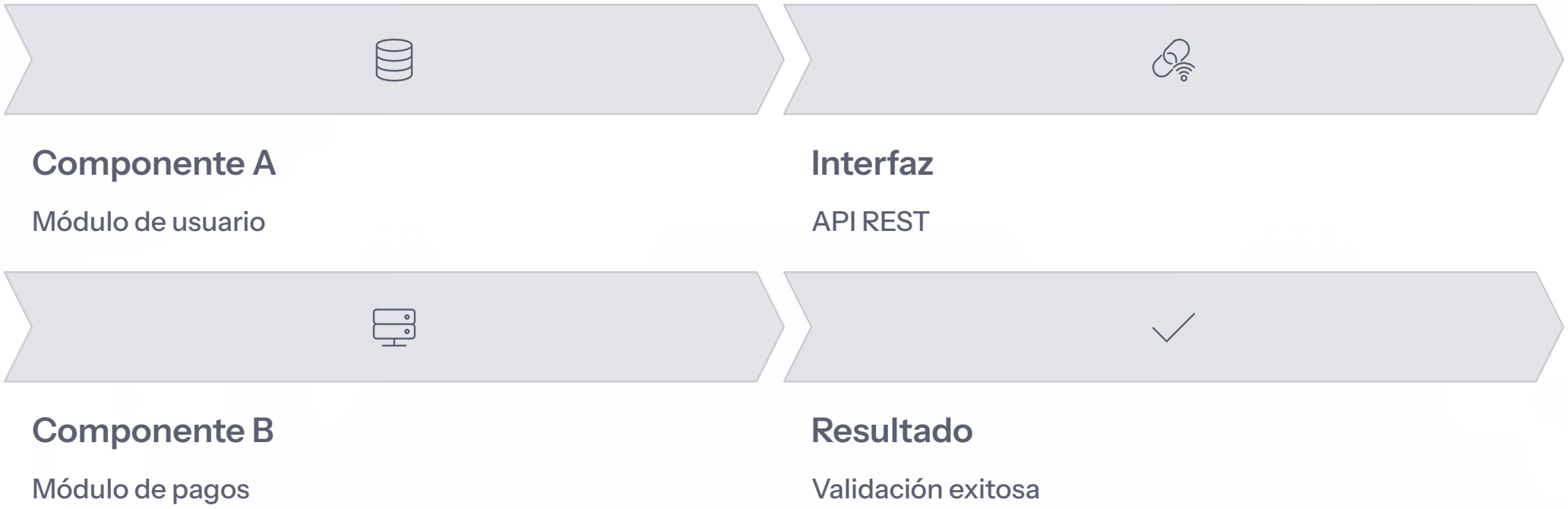
Sesión 3: Integration Testing

Objetivos de Aprendizaje

- Comprender la diferencia entre unit y integration testing
- Diseñar pruebas de integración efectivas
- Identificar interfaces y puntos de conexión
- Aplicar mocking y stubbing para dependencias

Desarrollo de Conceptos

El **testing de integración** verifica que múltiples componentes trabajen juntos correctamente. A diferencia de las pruebas unitarias, estas pruebas combinan módulos y validan la comunicación entre ellos.



Caso Práctico: E-commerce

Amazon utiliza pruebas de integración para validar el flujo completo de compra: desde el carrito (Componente A) hasta el procesamiento de pago (Componente B) y generación de orden (Componente C). Prueban casos como: stock suficiente, stock insuficiente, método de pago fallido, y timeout en conexión. Cada prueba simula el escenario completo y verifica que todos los componentes se comuniquen correctamente.

Preguntas de Reflexión

1. ¿Por qué no reemplazar todas las pruebas unitarias con pruebas de integración si validan el sistema completo?
2. ¿Cómo afecta el mocking el realismo de las pruebas de integración?
3. ¿Qué estrategia usarías para pruebas de integración en microservicios?

Conclusiones

Las pruebas de integración son esenciales para garantizar que los componentes individuales funcionen conjuntamente, descubriendo errores que las pruebas unitarias no detectan.

Made with GAMMA

Sesión 4: Test Driven Development (TDD)

Objetivos de Aprendizaje

- Aplicar la metodología TDD en desarrollo
- Comprender el ciclo Red-Green-Refactor
- Identificar ventajas y desafíos de TDD
- Diseñar código testable desde el inicio

Desarrollo de Conceptos

Test Driven Development invierte el proceso tradicional: primero se escriben las pruebas, luego el código que las hace pasar, y finalmente se refactoriza. Esta metodología produce código más robusto y mantenible.

Implementar Código

Escribe mínimo código para pasar la prueba

Escribir Prueba

Define comportamiento esperado antes de implementar

Refactorizar

Mejora código manteniendo funcionalidad

Caso Práctico: Desarrollo en Google

En **Google**, equipos como el de Gmail aplican TDD rigurosamente. Antes de añadir una nueva función de borrado seguro, escriben pruebas que verifican: borrado inmediato, recuperación en 30 días, y limpieza permanente después de 60 días. El código se implementa iterativamente hasta que todas las pruebas pasan. Resultado: tasa de bugs 40% menor en nuevas funcionalidades.

Preguntas de Reflexión

1. ¿Qué desafíos presenta TDD en proyectos existentes sin pruebas previas?
2. ¿Cómo afecta TDD la velocidad inicial de desarrollo versus mantenibilidad?
3. ¿Es TDD aplicable a todos los tipos de desarrollo (frontend, backend, ML)?

Conclusiones

TDD mejora calidad y mantenibilidad del código, aunque requiere cambio cultural en equipos. Es más efectivo cuando se aplica desde el inicio del proyecto.

Made with GAMMA

Sesión 5: Functional Testing - End-to-End

Objetivos de Aprendizaje

- Definir pruebas funcionales de extremo a extremo
- Diseñar casos de prueba de escenarios completos
- Utilizar herramientas como Selenium, Cypress o Playwright
- Diferenciar entre testing funcional y no funcional

Desarrollo de Conceptos

Las **pruebas funcionales end-to-end** simulan la experiencia completa del usuario, desde la interfaz hasta la base de datos. Verifican que el sistema funcione como un todo integrado.

Caso Práctico: Netflix

Netflix ejecuta pruebas E2E para el flujo de registro y reproducción: usuario crea cuenta → selecciona plan → proporciona pago → busca contenido → reproduce video. Cada paso valida interfaces web, APIs, servicios de pago y streaming. Ejecutan 200+ pruebas E2E diariamente, con capturas de pantalla automáticas para verificar UI.

Preguntas de Reflexión

1. ¿Por qué Netflix necesita capturas de pantalla si ya valida funcionalidad?
2. ¿Cuál es el balance entre cobertura de pruebas E2E y velocidad de ejecución?
3. ¿Cómo manejarías pruebas E2E con dependencias externas (redes sociales, APIs de terceros)?

Conclusiones

Las pruebas E2E son cruciales para validar experiencia de usuario completa, aunque son más lentas y complejas. Deben complementarse con pruebas unitarias e integración.

Made with GAMMA

Sesión 6: Regression Testing

Objetivos de Aprendizaje

- Comprender el propósito del testing de regresión
- Seleccionar casos de prueba para regresión
- Automatizar pruebas de regresión efectivamente
- Gestionar suites de pruebas de regresión

Desarrollo de Conceptos

El **testing de regresión** verifica que cambios en el código no rompan funcionalidad existente. Es crítico en desarrollo ágil donde el código cambia frecuentemente.

Selección de Pruebas

Identifica casos afectados por cambios. Usa análisis de impacto para priorizar.

Ejecución Automática

Pruebas de regresión deben ejecutarse automáticamente en cada commit o build.

Análisis de Resultados

Identifica qué pruebas fallaron y si el cambio causó el fallo o hay falsos positivos.

Caso Práctico: Facebook

Facebook ejecuta más de 100,000 pruebas de regresión con cada cambio. Antes de desplegar cualquier actualización, valida que todas las funcionalidades previas (news feed, chat, publicaciones) sigan funcionando. Utilizan técnicas de selección inteligente para ejecutar solo pruebas relevantes según cambios realizados, reduciendo tiempo de 8 horas a 45 minutos.

Preguntas de Reflexión

1. ¿Cómo determina Facebook qué pruebas ejecutar según cada cambio?
2. ¿Qué estrategia usarías si no puedes ejecutar todas las pruebas en cada commit?
3. ¿Cómo balancearías cobertura de regresión con velocidad de desarrollo?

Conclusiones

Las pruebas de regresión protegen inversión en funcionalidad existente. La automatización es esencial para ejecutarlas frecuentemente sin impactar velocidad de desarrollo.

Made with GAMMA

Sesión 7: Smoke y Acceptance Testing

Objetivos de Aprendizaje

- Distinguir entre smoke y acceptance testing
- Diseñar pruebas smoke para builds rápidos
- Comprender criterios de aceptación
- Participar en reviews de aceptación con stakeholders

Desarrollo de Conceptos

Smoke testing verifica que funcionalidades críticas funcionan antes de testing profundo. **Acceptance testing** valida que el software cumple requisitos del negocio.

Smoke Testing

"Prueba de humo" - verifica que el build es estable antes de continuar testing.

- Ejecuta en minutos, no horas
- Cubre funcionalidades críticas
- Decisión: continuar o descartar build

Acceptance Testing

Valida que software cumple requisitos del negocio y está listo para producción.

- Define usuarios finales o PO
- Criterios objetivos de éxito
- Último paso antes de release

Caso Práctico: Microsoft Office

Microsoft aplica smoke testing en cada build de Office: verifica que aplicación inicia, documento se abre, y se puede guardar. Si falla, el build se descarta automáticamente. Para acceptance testing, invitan usuarios beta a probar nuevas versiones durante semanas, validando que funcionalidades como "Insertar imagen" o "Revisión ortográfica" cumplen expectativas.

Preguntas de Reflexión

1. ¿Por qué Microsoft separa smoke de acceptance testing en lugar de combinarlas?
2. ¿Qué funcionalidades considerarías "críticas" para smoke testing en un sistema bancario?
3. ¿Cómo manejarías aceptación cuando stakeholders tienen requisitos contradictorios?

Conclusiones

Smoke testing protege tiempo de testing profundo; acceptance testing asegura que el software resuelve problema del negocio. Ambos son filtros críticos.

Made with GAMMA

Sesión 8: Performance Testing

Objetivos de Aprendizaje

- Comprender tipos de pruebas de performance
- Identificar métricas clave de rendimiento
- Configurar herramientas como JMeter o Gatling
- Analizar resultados y identificar cuellos de botella

Desarrollo de Conceptos

El **performance testing** evalúa velocidad, estabilidad y escalabilidad bajo carga. Incluye: carga normal, picos, estrés y escalabilidad.

Load Testing

Verifica comportamiento bajo carga esperada. Ej: 1000 usuarios concurrentes en e-commerce durante día normal.

Stress Testing

Sobrepasa límites para encontrar punto de quiebre. Ej: aumentar usuarios hasta que servidor colapse.

Scalability Testing

Valida que sistema escala horizontalmente. Ej: agregar servidores reduce tiempo de respuesta proporcionalmente.

Caso Práctico: Instagram

Instagram ejecuta pruebas de carga antes de eventos masivos como Super Bowl. Simulan 5 millones de usuarios publicando fotos simultáneamente. Métricas: tiempo de respuesta < 2s, tasa de errores < 1%, CPU < 80%. Identificaron cuello de botella en servicio de redimensionamiento de imágenes, que solucionaron añadiendo instancias y cache.

Preguntas de Reflexión

1. ¿Por qué Instagram prioriza tiempo de respuesta sobre 100% disponibilidad?
2. ¿Cómo diferencias entre testing de carga en pre-producción versus monitoreo en producción?
3. ¿Qué métricas usarías para un sistema de trading versus red social?

Conclusiones

El performance testing identifica cuellos de botella antes de producción. Es crítico para sistemas con usuarios concurrentes altos o requisitos de baja latencia.

Made with GAMMA

Sesión 9: Métricas de Software y CI/CD

Objetivos de Aprendizaje

- Calcular y interpretar métricas de testing
- Comprender conceptos de CI/CD
- Configurar pipelines de integración continua
- Utilizar servicios cloud para testing automatizado

Desarrollo de Conceptos

Las **métricas** cuantifican calidad y eficiencia del proceso de testing. **Continuous Integration** integra cambios frecuentemente; **Continuous Delivery** automatiza despliegue.

85%

Cobertura de Código

Porcentaje de líneas ejecutadas en pruebas

99.9%

Disponibilidad

Tiempo sistema operativo versus planificado

2h

Tiempo Medio de Recuperación

Tiempo para restaurar servicio tras fallo

15

Deploys Diarios

Frecuencia de despliegues a producción

Caso Práctico: GitHub

GitHub utiliza CI/CD con pipelines en cada commit: lints → unit tests → integration tests → build → deploy a staging → acceptance tests. Si todo pasa, se despliega automáticamente a producción. Ejecutan 500+ deploys diarios. Cloud: AWS para infraestructura, GitHub Actions para orquestación. Métricas: 99.95% uptime, 95% cobertura, 30min tiempo medio de recuperación.

Preguntas de Reflexión

1. ¿Por qué GitHub puede hacer 500 deploys diarios sin romper producción?
2. ¿Qué riesgos tiene despliegue automático sin aprobación humana?
3. ¿Cómo balancearías velocidad de CI/CD con seguridad y calidad?

Conclusiones

CI/CD automatiza integración y despliegue, reduciendo riesgo de errores. Las métricas guían mejoras en proceso. Cloud permite escalar infraestructura de testing.

Made with GAMMA